

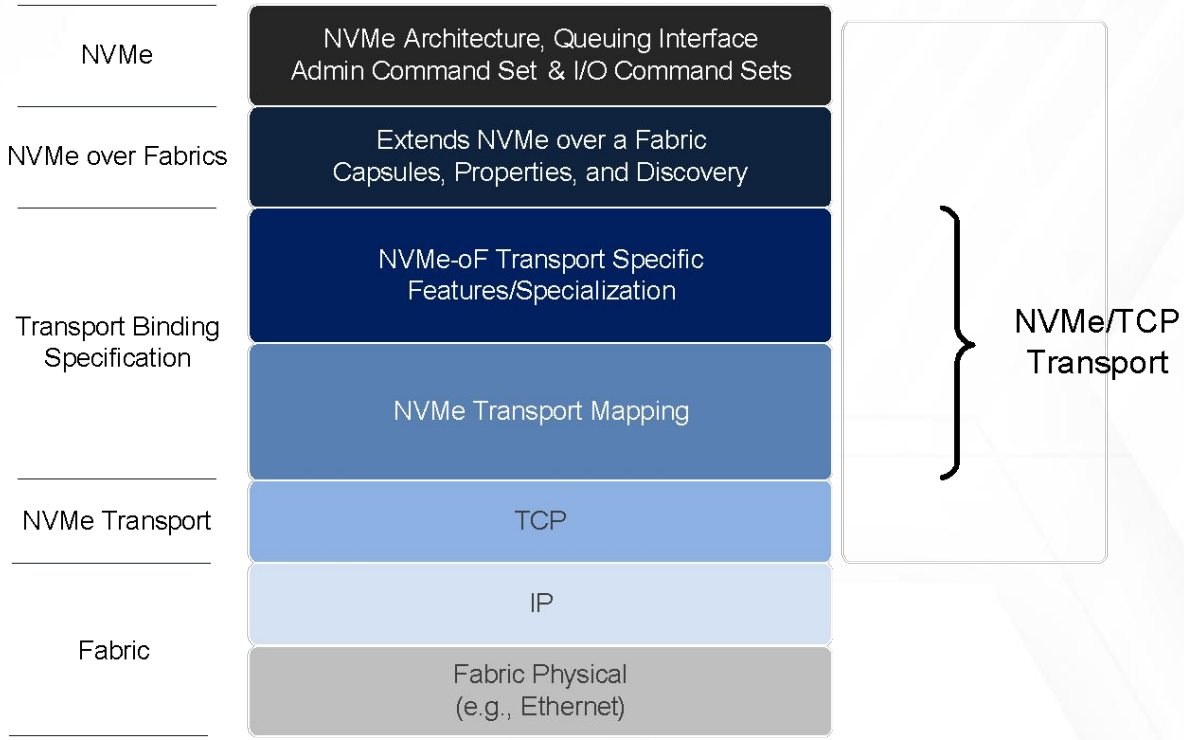
Performance improvements for NVMe/TCP

Netdev 0x14, 2020

Sagi Grimberg, Lightbits

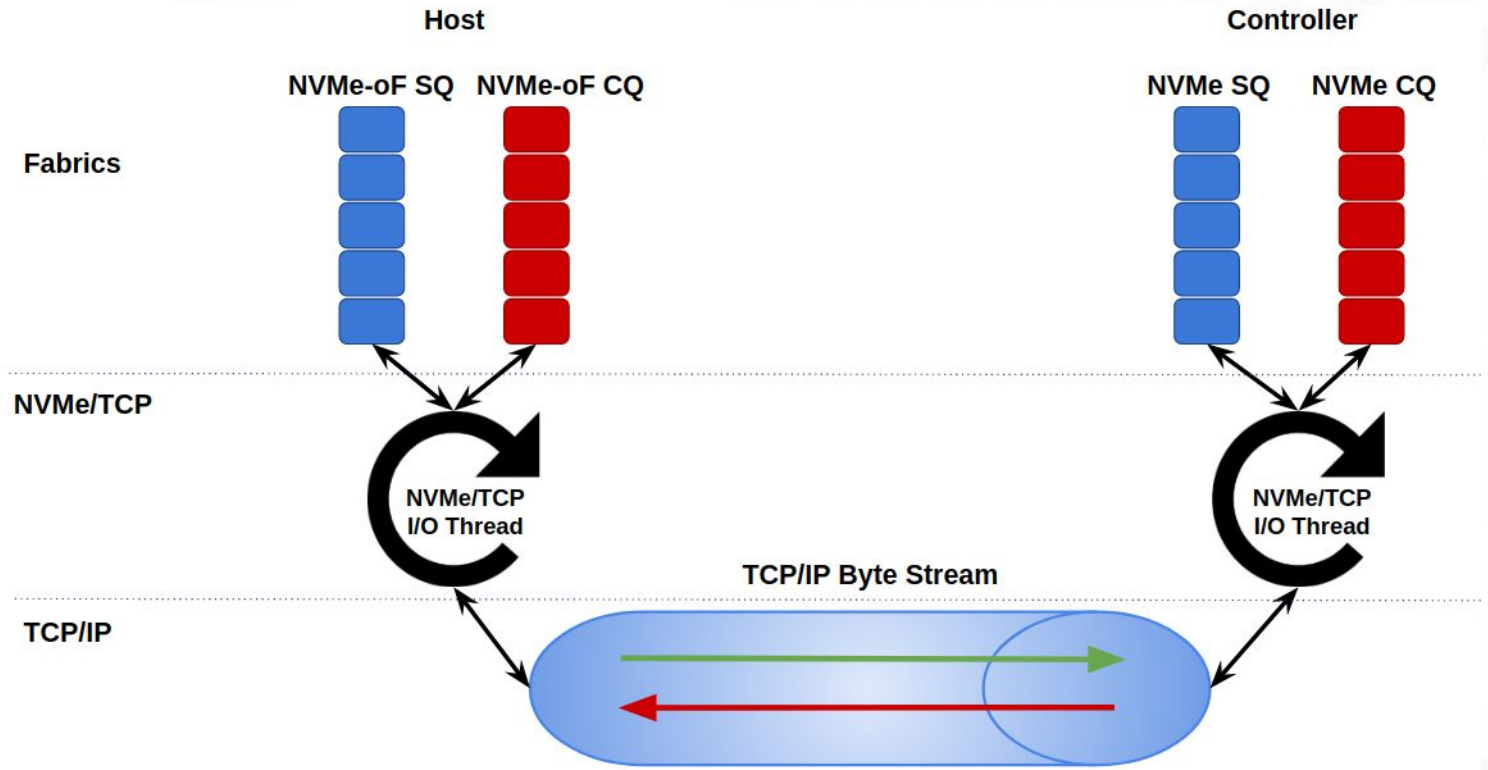
NVMe/TCP short Intro

- NVMe/TCP is the standard transport binding to run NVMe on top of standard TCP/IP networks
- Standard NVMe multi-queue interface runs on top of TCP sockets
- Same NVMe command set, encapsulated over NVMe/TCP PDUs



NVMe/TCP short Intro

- Each NVMe queue-pair is mapped to a bidirectional TCP socket
- Commands and data-transfer are processed by a bounded workqueue context
- Each NVMe queue normally mapped to a dedicated CPU core



Latency contributions

- **Serialization** - lightweight, only on a per-queue basis (and hctx, sockets etc) - scales pretty well
- **Context Switching** - 2 at a minimum contributed by the driver
- **Memory copy** - only on RX, not a huge contributor (sometimes is on high load)
- **Interrupts** - Definitely impactful, LRO/GRO/Adaptive-moderation can mitigate a bit, but latency is less consistent
- **socket overhead** - Exists, but not huge, mostly around small size RX/TX
- **Affinitization** - definitely a contributor if not affinitized correctly
- **Cache pollution** - Has some, not excessive
- **Head-of-Line blocking** - Apparent in mixed workloads

Asynchronous Direct I/O (Host side)

1. user issues issues direct file/block I/O (aio or io_uring) — **syscall**
    a. Then *maybe* issued directly to the driver, maybe to an I/O scheduler — ***May* Context-Switch**
2. nvme_tcp_queue_rq prepares NVMe/TCP PDU and place it in a queue — **Context-Switch**
3. nvme_tcp_io_work context picks up I/O and process it
4. I/O completes, controller sends back data/completion to the host
5. NIC generates interrupt — **Soft-IRQ**
6. NAPI is triggered
7. nvme_tcp_data_ready is triggered — **Context-Switch**
8. nvme_tcp_io_work context is triggered, processing and completing the I/O
9. user context completes (via io_getevents)

Mixed workloads optimizations

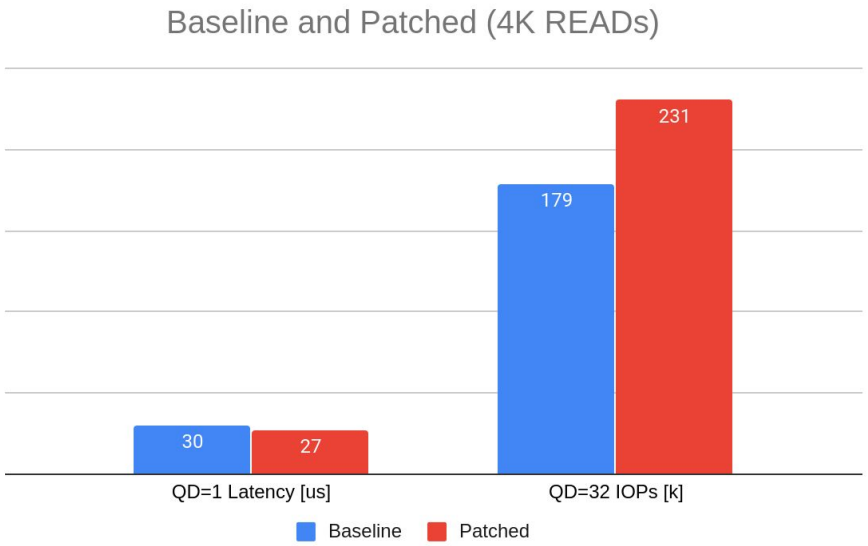
- Linux block layer allows for multiple queue maps
  - Default: normal set of HW queues
  - Read: Dedicated queues for Reads
  - Poll: Dedicated queues for polling application and RWF_HIPRI I/O
- This helps eliminate Head-of-Line blocking of small TCP payloads (small reads) TCP bulk transfers generated (large writes)
- Added support for multiple queue maps and plugging into the generic block layer:

Test: 16 readers issuing synchronous 4K reads, 1 unbound writer issuing 1M writes @ QD=32

	READ IOPs [k]	READ Average Latency [us]	READ 99.99% tail latency [us]
Baseline	80.4	396	14222
Patched	171	181.5	1811

Affinity improvements

- Linux grew capability to split different I/O types to different queue maps
- optimize queue io_cpu assignment for multiple queue maps
  - In case there are multiple queue maps (read/default/polling) use separated alignment
  - The use-case is that often the application will want multiple queues of different types
  - Instead of calculating the alignment of all queues altogether align each queue map individually



Low QD latency optimizations - TX path

- Eliminate NVMe/TCP context switch when queuing a request
  - Prepare NVMe/TCP and process directly from nvme_tcp_queue_request
  - Network send might_sleep, so need to convert hctx locking to srcu
  - Need to serialize two contexts of the same queue (introduce a mutex)
    - Do that only if the queue is empty
    - Do it only if the queue mapped CPU matches the running cpu

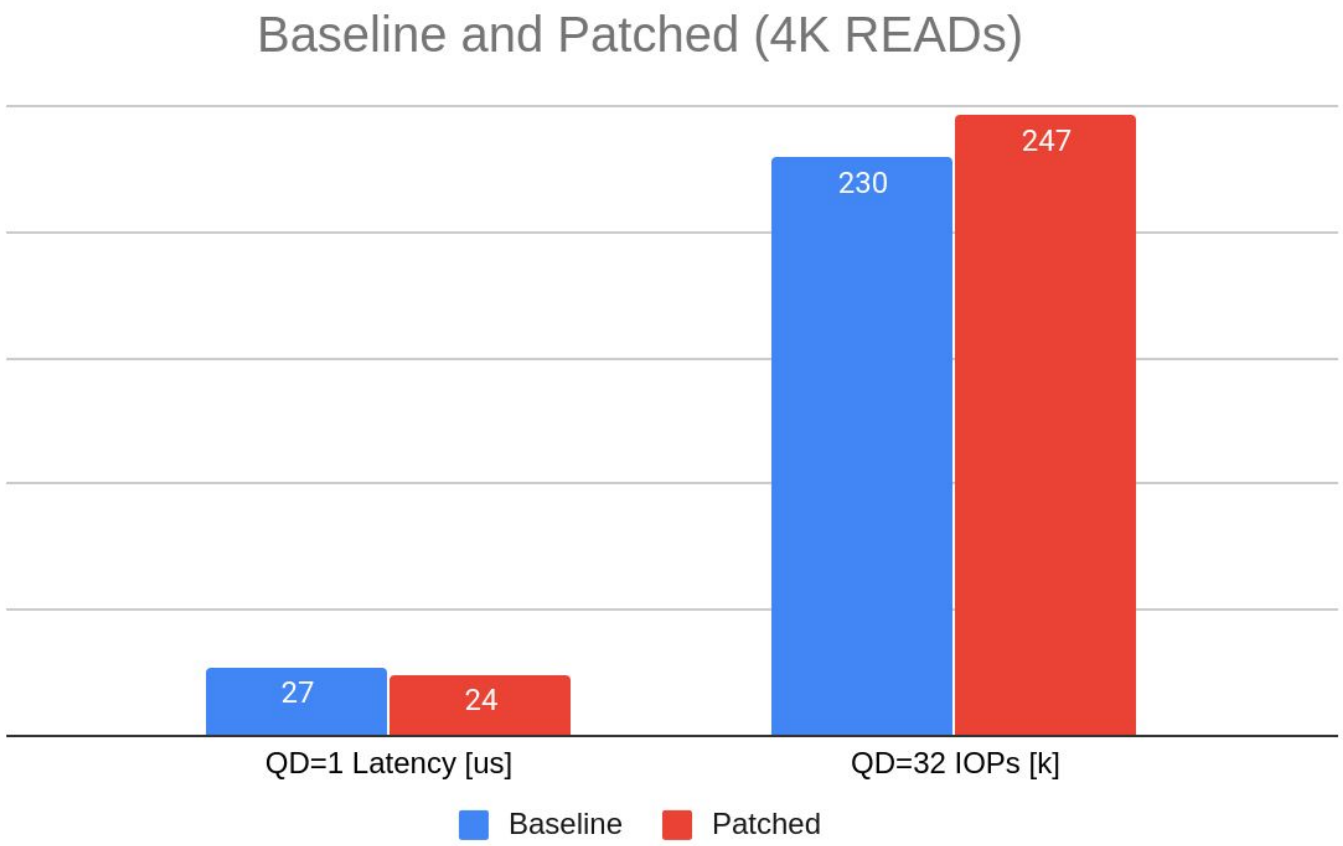
Low QD latency optimizations - RX Path

- Linux grew a polling interface for latency sensitive I/O
  - Submit with RWF_HIPRI
  - Poll for completion (also via io_uring IORING_SETUP_IOPOLL)
- We add nvme_tcp_poll and plug it into blk_poll interface
  - Add dedicated queues for polling (connect options)
  - nvme_tcp_poll calls sk_busy_loop

- Skip RX data_ready context switch if application is polling at the same time
  - Mostly true if NIC moderation is working well
  - If device can hold off interrupts more aggressively it works very well

Low QD latency optimizations - RX Path

- Measurements done for a Single core running fio



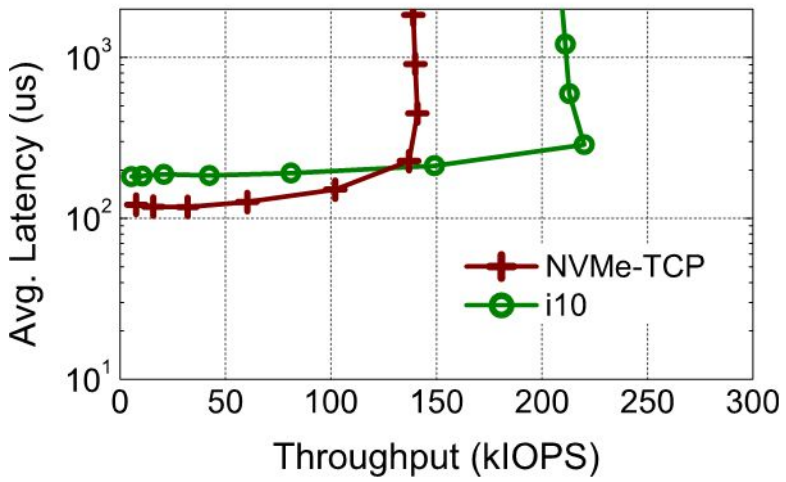
High QD latency optimizations - Batching

- If a high QD is building up on the queue, we want to leverage that information
- The block layer now indicates to the driver that the request being queued is not the last one and more is coming (bd->last indicator)

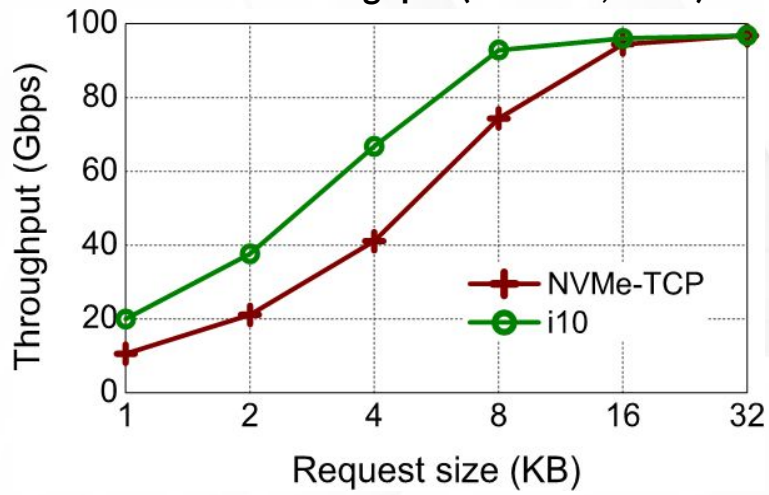
- Modify the driver send queue from list (protected by a spinlock) to a lockless list
  - I/O thread pulls from list in batches, has a better view of what is coming
  - Schedule I/O thread only when the “last in batch” arrives...
- Optimize network MSG flags based on this information: MSG_MORE, MSG_SENDPAGE_NOTLAST (and MSG_OER if last in batch)
- Improve batching support in blk-mq in case of I/O schedulers [Ming Lei]
- Implement an optimized batching scheduler for TCP stream based storage devices
  - i10 paper [Jaehyun Hwang, Qizhe Cai Ao Tang, Rachit Agarwal Cornell University]

High QD latency optimizations - Batching

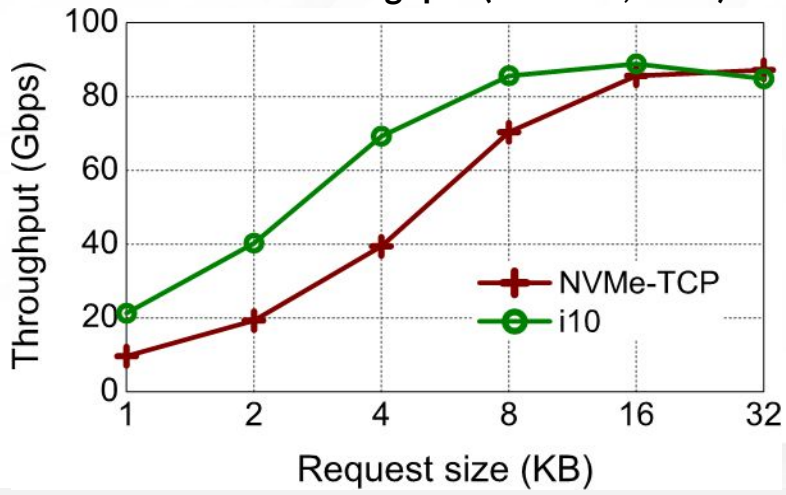
4KB READ IOPs (Single Core, SSD)



READ Throughput (16 cores, RAM)



WRITE Throughput (16 cores, RAM)





Questions ?

Netdev 0x14, 2020

Sagi Grimberg, Lightbits